

Nonlinear Model Predictive Control for Energy Optimization of Hybrid Electric Vehicles on Repeated Routes via Value Function Approximation

Suyong Park, *Student Member, IEEE*, Solyeon Kwon, *Student Member, IEEE*,
Youngki Kim, *Senior Member, IEEE*, and Kyoungseok Han, *Member, IEEE*

Abstract—Hybrid electric vehicles (HEVs) utilize dual power sources requiring optimal energy distribution through sophisticated energy management systems (EMS). While nonlinear model predictive control (MPC) enables real-time energy optimization using short-term information, it suffers from prediction horizon limitations that prevent utilization of complete driving route information. This paper addresses this limitation by proposing approximated value function MPC (V-MPC) that incorporates reinforcement learning (RL) to approximate complex value function (VF) calculations using speed profile characteristics from repeated driving routes. The results show that V-MPC exhibits fuel consumption deviations of 3.15-5.70% relative to the optimal dynamic programming (DP) solution, while conventional MPC shows significantly larger deviations of 7.22-10.24%. Real-time validation using Autonomie software confirmed real-time applicability, achieving approximately 3.96% fuel consumption reduction compared to rule-based (RB) control while maintaining charge-sustaining (CS) mode operation under repeated driving scenarios. The proposed V-MPC achieves real-time energy optimization for repeated driving routes without requiring complete route information in advance.

Index Terms—Hybrid electric vehicles, energy management strategy, model predictive control, value function approximation, reinforcement learning.

I. INTRODUCTION

ELECTRIC vehicles (EVs) have garnered considerable attention as a result of stringent regulatory policies targeting internal combustion engines (ICEs) across multiple countries. Furthermore, regenerative braking systems and high-efficiency electric motors facilitate substantial reductions in overall energy consumption [1], [2]. However, the complete transition from ICE vehicles to EVs remains challenging due to battery performance degradation under varying temperature conditions and limitations in charging infrastructure [3], [4].

Recently, hybrid electric vehicles (HEVs) are being considered as a balanced alternative between EVs and ICE vehicles

[5], [6]. HEVs utilize multiple power sources to provide the most efficient powertrain configuration based on driving conditions, thereby simultaneously optimizing fuel economy and performance. Therefore, an energy management system (EMS) is essential for the integrated management of these complex power sources, which not only ensures system safety by preventing overcharge and overdischarge, but also maximizes overall system efficiency [7], [8].

To achieve this optimal energy management, rule-based (RB) and optimization methods are the most widely adopted EMS approaches [9]. RB approaches are characterized as real-time control strategies that operate on predefined rules established through expert knowledge and characteristics of the powertrain. In [10], the HEV operating modes and points were obtained through test data and control strategies were reconstructed according to system states such as the power of wheel. However, RB make it difficult to achieve optimal solutions under all operating conditions and cannot guarantee optimality [11].

Subsequently, optimization methods are increasingly preferred for EMS since they guarantee both global and local optimality. Optimization-based control methods can be classified into two main categories: global and real-time optimization approaches. Dynamic programming (DP) can ensure global optimality when complete traffic information is available. In [12], discrete-time optimal control problems are addressed using Bellman's principle to optimize the efficiency of parallel HEVs. As a result, total fuel consumption is minimized, and an optimal trajectory for state-of-charge (SOC) is achieved. However, obtaining complete traffic information in real time remains challenging due to the uncertainties of real-world driving conditions and significant computational requirements. In contrast, nonlinear model predictive control (MPC) can be considered for real-time optimization [13], [14]. When short-term prediction information is available, EMS can be developed by explicitly incorporating various physical constraints into the optimization problem and minimizing the objective function [15], [16]. Although these methods provide assured optimality, they are constrained by their dependence on anticipated driving conditions for control optimization.

Most recently, reinforcement learning (RL) has been increasingly used to improve EMS performance [17]. RL interacts with the environment to learn and discover optimal strategies without relying on predefined rules or mathematical models. RL in EMS can explore the best solution with

This work was supported by the Korea Agency for Infrastructure Technology Advancement (KAIA) grant funded by the Ministry of Land, Infrastructure and Transport (MOLIT) of the Korean government (Grant RS-2026-25522607), and by the Regional Innovation System & Education (RISE) program through the Seoul RISE Center, funded by the Ministry of Education (MOE) and the Seoul Metropolitan Government (Grant 2026-RISE-01-027-04).

Suyong Park, Solyeon Kwon and Kyoungseok Han are with the Department of Automotive Engineering, Hanyang University, Seoul, 04763, Republic of Korea (e-mail: suyongpark:solyeonk:kyoungsh@hanyang.ac.kr).

Youngki Kim is with the Department of Mechanical Engineering, University of Michigan-Dearborn, Dearborn, MI 48128 USA (e-mail: youngki@umich.edu).

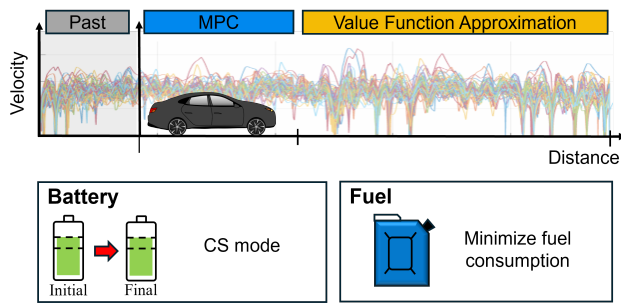


Fig. 1. V-MPC architecture for HEV under repeated driving route.

complex driving cycle to minimize fuel consumption [18]. In contrast to optimization, RL can directly apply learned control inputs based on vehicle states without computational burden. However, learning an effective policy requires a lot of trial and error iteration. Since this is not feasible with real vehicles, the reliance on simulators becomes necessary. Furthermore, it is difficult to prove the reliability of RL under the stringent safety standards and certification processes required in the automotive industry. Therefore, RL needs hybrid approaches that combine existing RB or optimization methods.

To address the respective limitations of the optimization method and RL, the combination of MPC and RL can be considered. In [19], double Q-learning (DQL) is proposed to learn optimal power distribution strategy for HEV. Action value function from trained DQL is considered to solve the rolling optimization process in MPC for HEV. Alternatively, value function (VF) approximation methods using RL can be employed to improve MPC performance [20], [21]. VF represents the total future rewards from following a policy in a given state. This overcomes short prediction horizon limitations of MPC by evaluating the potential for favorable future outcomes based on MPC results. In this work, we adopt combination of MPC and VF (V-MPC) to overcome limitation of short-term prediction information.

Extensive research has been conducted on VF approximation, where [20] used quadratic function approximation of DP-derived cost-to-go maps for MPC horizon optimization. However, the offline computation of state-dependent VF for specific scenarios makes it challenging to establish general state value definitions for other scenarios. Similarly, existing research [22] develops an MPC for a parallel plug-in HEV using a VF approach, where VF is derived through machine learning. The results showed that energy consumption was minimized through VF in various scenarios. However, to create high-quality data for regression models, complex modeling must be considered within the DP framework. Alternatively, VF considered in the prediction horizon is a crucial value for deriving MPC's optimal solution, and RL can learn optimal values through environmental interaction without requiring complex models. In [23], they propose MPC using RL to tune MPC scheme when using a wrong model. MPC can serve as a function approximator within the RL framework, while RL learns MPC's tuning parameters including weight factors and model matrices for linear quadratic regulator

problems. Although wrong model was employed in MPC, the performance of MPC's prediction horizon optimization was enhanced through RL-based tuning of MPC model parameters.

Since VF can be used as a tuning parameter in MPC prediction horizon optimization and can overcome short-term limitations by evaluating future values, we adopted a VF approximation method through interaction between MPC and RL for HEVs operating on repeated driving routes, such as fixed commute paths or bus routes [24]. As shown in Fig. 1, the MPC calculates an optimal power distribution to minimize energy consumption. When VF is approximated offline using historical data, it effectively evaluates future results for similar driving patterns. This can address MPC's short-term prediction limitations. However, driving in the real world involves complex scenarios that make analytical VF computation difficult. Therefore, we use RL to approximate the VF under various driving scenarios. RL can learn optimal policies through environmental interaction without requiring complex mathematical models. The approximated VF is integrated into MPC's optimization process as a terminal cost function. This enables the controller to consider both immediate efficiency and long-term performance. V-MPC balances trade-offs between battery and fuel consumption throughout the prediction horizon, while maintaining charging sustaining (CS) mode operation. The main contributions of this study are as follows:

- We developed V-MPC to minimize energy consumption in HEVs operating on repeated driving routes. MPC is a real-time optimization method that enables optimal power distribution using short-term predicted information without requiring complete driving information compared to DP. However, using conventional MPC methods makes it difficult to guarantee global solutions due to optimization that relies on short-term information. Therefore, we improved MPC performance by applying VF approximation that evaluates the value of MPC states in repetitive driving pattern scenarios.
- We approximated complex VF calculations through RL. The VF considered in MPC's prediction horizon optimization is a critical factor for finding optimal MPC solutions. Therefore, RL can be useful to learn VF through interaction with MPC. The approximate VF using RL minimizes fuel consumption and regulates battery usage.
- The V-MPC was trained offline and tested real-time on various scenarios and complex HEV models to verify real-time performance. The VF values learned through RL were applied to the HEV model in Autonomie software, demonstrating the versatility and practical applicability of the MPC-RL hybrid approach.

The remainder of this paper is organized as follows: Section II introduces vehicle dynamics and powertrain system modeling. Section III presents the optimal control problem for the HEV. Section IV presents V-MPC, and VF approximation using RL. Section V analyzes simulation results compared with baseline approaches to evaluate V-MPC performance. Finally, Section VI concludes the paper.

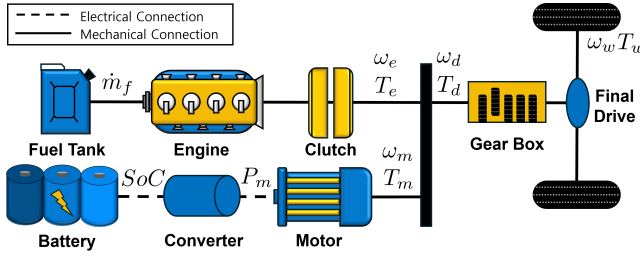


Fig. 2. Schematic of pre-transmission mild PHEV configuration.

II. MODELING

In this section, we introduce vehicle longitudinal dynamics and powertrain modeling for parallel hybrid electric vehicles (PHEVs). As shown in Fig. 2, the schematic of a PHEV is depicted. PHEVs can operate using multiple energy sources, including fuel and battery power. Given a driver's acceleration command, the required wheel speed and torque can be determined. Subsequently, optimal power distribution between powertrain components such as the engine and motor must be achieved while minimizing energy consumption. Detailed modeling will be introduced in the following subsection.

A. Vehicle longitudinal dynamics

In this work, we consider longitudinal dynamics in the space domain, because it is more rational for driving patterns to be specified according to driving distance rather than time [25]. The dynamics in the discrete space domain are expressed as follows:

$$\frac{1}{2}mv^2(d+1) - \frac{1}{2}mv^2(d) = (F_t - F_a - F_g - F_f)\Delta d, \quad (1)$$

where m is the vehicle mass, v is the velocity in the space domain, d is the travel distance, and Δd is the sampling distance. Considering the time-varying nature of vehicle mass m due to fuel consumption, the instantaneous mass can be expressed as:

$$m(d) = m_0 - \sum_{d_0}^d \Delta m_{f,d} \Delta d \quad (2)$$

where m_0 is initial vehicle mass including fuel, d_0 is initial travel distance, and $\Delta m_{f,d}$ is the instantaneous fuel consumption rate.

The forces F_t , F_a , F_g , and F_f are the tractive force, aerodynamic resistance, gravitational resistance, and rolling resistance, respectively. F_t is determined from driver acceleration and resistance forces. The resistance forces F_a , F_g , and F_f are formulated as follows:

$$F_a = \frac{1}{2}\rho AC_D v^2, \quad (3)$$

$$F_g = mg \sin(\theta), \quad (4)$$

$$F_f = fmg \cos(\theta), \quad (5)$$

where ρ is the air density, A is the frontal area of the vehicle, C_D is the drag coefficient, g is the gravitational acceleration, f is the rolling resistance coefficient, and θ is the road

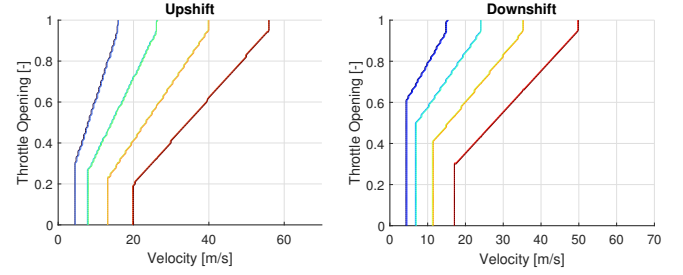


Fig. 3. Gear upshift and downshift map.

slope. Considering that mild-type PHEVs have limited motor specifications, the application of hybrid mode is restricted in areas with steep slopes. In this study, we assume that $\theta = 0$. The traction force F_t can be expressed by:

$$F_t = \frac{T_w}{r_w}, \quad (6)$$

$$T_w = T_d i_g i_0 + T_b \approx T_d i_g i_0, \quad (7)$$

where T_w is wheel torque, T_d is the demanded torque, T_b is braking torque, i_g is the gear ratio, i_0 is the final drive ratio, and r_w is the wheel radius. In this study, we assume that the electric motor alone handles all necessary braking torques. Therefore, we set T_b to zero because friction brakes are not needed when the motor provides sufficient braking. T_d can be calculated as follows:

$$T_d = uT_e + (1 - u)T_m, u \in [0, 1], \quad (8)$$

where u is the torque split ratio, T_m and T_e represent motor and engine torques, respectively. Our EMS uses u to distribute T_m and T_e values, which are important to optimize fuel and battery consumption.

The demanded angular velocity ω_d at the gearbox input can be calculated by:

$$\omega_w = \frac{v}{r_w}, \quad (9)$$

$$\omega_d = \omega_w i_g i_0, \quad (10)$$

where ω_w is wheel speed. In this work, we ignore wheel lock or tire slip that can occur during sudden braking and acceleration in (9). The gear ratio i_g is calculated using the predefined map in Fig. 3.

B. Energy consumption model in space domain

In this section, we introduce energy consumption model in space domain. The energy consumption model of the engine and motor can be defined using the engine speed w_e and torque T_e , and the motor speed w_m and torque T_m , respectively. T_m and T_e are calculated by (8) using distribution ratio u . The variables w_e and w_m can be expressed as follows:

$$\omega_e = \omega_m = \omega_d. \quad (11)$$

We assume that the engine and the motor are rigidly coupled, which implies that w_m and w_e are always equal. As shown in Fig. 4, motor power and fuel consumption maps

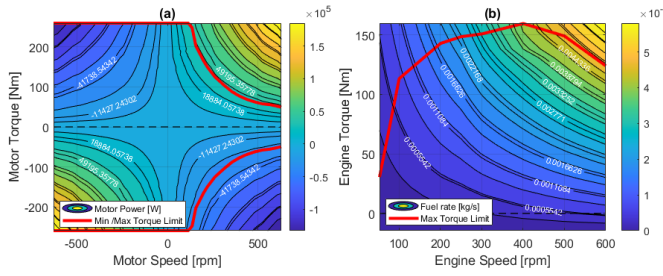


Fig. 4. Energy consumption model in time domain. (a) motor power map, (b) fuel consumption map.

TABLE I
VEHICLE AND POWERTRAIN SIMULATION PARAMETERS

Parameter	Symbol	Value
<i>Battery Parameters</i>		
Battery capacity	C_b	5.943 Ah
Initial SOC	SOC_i	0.6
<i>Motor Parameters</i>		
Max torque	$T_{m,max}$	259.7 Nm
Min torque	$T_{m,min}$	-400 Nm
Max speed	$\omega_{m,max}$	628.3 rad/s
<i>Engine Parameters</i>		
Max torque	$T_{e,max}$	140.1 Nm
Max speed	$\omega_{e,max}$	523.6 rad/s
Fuel heating value	LHV_{fuel}	42.86 MJ/kg
<i>Vehicle Parameters</i>		
Vehicle mass	m_0	1730 kg
Frontal area	A	0.6753 m ²
Rolling resis. coeff.	f	0.01
Wheel radius	r_w	0.3014 m
Final drive ratio	i_0	4.44
Gear ratios	i_g	[2.56, 1.55, 1.02, 0.72, 0.52]

are depicted. Motor power P_m is calculated using T_m and ω_m , while fuel rate $\dot{m}_f$ is determined from T_e and ω_e based on given velocity and acceleration. Polynomial functions can be calculated using the curve-fitting tool in MATLAB. The following polynomial function is used to define engine fuel rate and motor power [26]:

$$\dot{m}_{f,t} = a_0 + a_1 T_e + a_2 \omega_e + a_3 T_e^2 + a_4 T_e \omega_e + a_5 \omega_e^2, \quad (12)$$

$$P_{m,t} = b_0 + b_1 T_m + b_2 \omega_m + b_3 T_m^2 + b_4 T_m \omega_m + b_5 \omega_m^2, \quad (13)$$

where $\dot{m}_{f,t}$ is the fuel consumption rate, and $P_{m,t}$ is motor power in time domain. a_i ($i \in [0, 5]$) are the coefficients of the fuel rate polynomial function, and b_i ($i \in [0, 5]$) are the coefficients of motor power polynomial function.

The discrete energy consumption model can be formulated

in time domain as follows:

$$\Delta m_{f,t} = \dot{m}_{f,t} \Delta t, \quad (14)$$

$$\Delta P_{m,t} = P_{m,t} \Delta t, \quad (15)$$

where Δt is sampling time.

Converting to the space domain by substituting $\Delta t = \Delta d/v$ as follows:

$$\Delta m_{f,d} = \dot{m}_{f,d} \Delta d, \quad (16)$$

$$\Delta P_{m,d} = P_{m,d} \Delta d, \quad (17)$$

where the space-domain rates are defined as:

$$\dot{m}_{f,d} = \frac{\dot{m}_{f,t}}{v}, \quad P_{m,d} = \frac{P_{m,t}}{v}. \quad (18)$$

Fuel consumption and motor power consumption are calculated at each sampling distance Δd . Battery power P_b can be calculated as follows:

$$P_b = P_{m,d}. \quad (19)$$

We assumed that auxiliary power consumers such as electrical accessories are neglected. The SOC can be calculated as follows:

$$\Delta SOC(s) = -\frac{V_{oc} - \sqrt{V_{oc}^2 - 4R_b P_b}}{2R_b C_b}, \quad (20)$$

where V_{oc} is open-circuit voltage, R_b is battery resistance and C_b is battery capacity. The energy consumption model is used to calculate the energy consumption of power distribution and is considered in the optimal control problem. The vehicle and powertrain parameters were obtained from Autonomie software and are presented in Table I.

III. OPTIMAL CONTROL PROBLEM

A. Optimal control problem for PHEV

In this section, we develop the optimal control formulation for the PHEV using a discrete space domain. Considering limited energy storage capacity of the PHEV system, proper power allocation is essential to achieve optimal performance while satisfying sustainability requirements of battery. The optimization problem can be stated as:

$$\min J = \sum_{d_0}^{d_f} L(x, u, d) \Delta d, \quad (21a)$$

subject to

$$x(d+1) = x(d) + f(x, u) \Delta d, \quad (21b)$$

$$x_{\min} \leq x(d) \leq x_{\max}, \quad (21c)$$

$$u_{\min} \leq u(d) \leq u_{\max}, \quad (21d)$$

where $x = SOC$ is the state, u is the control input, $f(x, u)$ is the state function, d_0 is the initial travel distance, and d_f is the final distance. x and u are constrained by (21c) and (21d). The objective function $L(x, u, d) = \dot{m}_{f,d}$ is minimized from d_0 to d_f . The control input $u \in [0, 1]$ governs the energy allocation between SOC and $\dot{m}_{f,d}$, which exhibit an opposing relationship. To manage the finite battery capacity of the PHEV, the SOC is maintained within the operational bounds of $SOC \in [0.5, 0.7]$.

By applying Bellman's principle and expanding (21a) up to $N-1$, the optimal control input u^* can be obtained as follows:

$$u^* = \underset{u}{\operatorname{argmin}} \left[\sum_{d=j}^{j+N-1} L(x, u, d) + \mathcal{J}^*(x^*(d+N), d+N) \right], \quad (22)$$

where $\mathcal{J}^*(x^*(d+N), d+N)$ represents the optimal cost-to-go function evaluated at the optimal state x^* , N is control horizon, and j is current distance. For notational simplicity, we denote $\mathcal{J}^*(x^*(d+N), d+N) := \mathcal{J}^*(x^*(d+N))$. When the optimal solution encompasses the optimal solutions of all constituent subproblems, the problem is said to conform to Bellman's principle of optimality [27]. The global optimal solution can be determined through DP given complete trajectory information [12].

B. Optimal value function

The optimal control input u^* can be obtained through DP that stores the cost-to-go $\mathcal{J}(\cdot)$ for all states and travel distances, solving via backward propagation. However, obtaining complete traffic information in real time is challenging due to uncertainties in real-world driving conditions. In this study, we adopt an approximation method to enable real-time optimization.

Using Taylor series expansion [28], $\mathcal{J}^*(SOC(d+N))$ around the optimal trajectory $SOC^*(d+N)$ can be expressed as follows:

$$\mathcal{J}^*(SOC(d+N)) = \mathcal{J}^*(SOC^*(d+N)) + \frac{\partial \mathcal{J}^*(SOC^*(d+N))}{\partial SOC} \cdot \Delta SOC + O(\cdot), \quad (23)$$

where $SOC^*(\cdot)$ denotes the optimal state trajectory and $\Delta SOC = SOC(\cdot) - SOC^*(\cdot)$ represents the state deviation. The final term $O(\cdot)$ characterizes the magnitude of higher-order terms, which can be neglected under the assumption of small ΔSOC values.

The target final SOC (SOC_f) is added and subtracted from ΔSOC . This accounts for the CS mode given the PHEV's limited battery capacity. (23) can be reconstructed by:

$$\begin{aligned} \mathcal{J}^*(SOC(d+N)) &\approx \mathcal{J}^*(SOC^*(d+N)) \\ &+ \frac{\partial \mathcal{J}^*(SOC(d+N-1))}{\partial SOC} \cdot (SOC_f - SOC(d+N-1)) \\ &+ \frac{\partial \mathcal{J}^*(SOC(d+N-1))}{\partial SOC} \cdot (SOC(d+N) - SOC_f). \end{aligned} \quad (24)$$

where $\mathcal{J}^*(SOC^*(d+N))$ are defined as:

$$\mathcal{J}^*(SOC^*(d+N)) = \mathcal{J}^*(SOC(d+N-1)), \quad (25)$$

Since $SOC^*(d+N)$ represents the optimal state that yields the minimum cost-to-go from $SOC(d+N-1)$, the equality $\mathcal{J}^*(SOC^*(d+N)) = \mathcal{J}^*(SOC(d+N-1))$ can be established.

Based on the Bellman optimality principle, the terms $\mathcal{J}^*(SOC^*(\cdot))$ and $(SOC_f - SOC(d+N-1))$ remain unaffected by the control input u , making them irrelevant to

the optimization process in (22). Consequently, (24) can be expressed as:

$$\mathcal{J}^*(SOC(d+N)) \approx \lambda \cdot (SOC(d+N) - SOC_f), \quad (26)$$

where

$$\lambda = \frac{\partial \mathcal{J}^*(SOC(d+N-1))}{\partial SOC}. \quad (27)$$

The state value $SOC(d+N)$ in relation to the control input u can be assessed through (27). By leveraging near-optimality properties in repetitive routes, we formulate an approximation to equation (26) through the following VF:

$$\hat{V}(SOC(d+N)) = \hat{\lambda} \cdot (SOC(d+N) - SOC_f), \quad (28)$$

where $\hat{V}(\cdot)$ represents the approximated VF and $\hat{\lambda}$ denotes the approximated λ . We establish and analyze the near-optimal properties of $\hat{V}(\cdot)$ for repetitive routes absent complete trip information. The function $\hat{V}(\cdot)$ serves as the terminal cost component in the MPC framework.

IV. V-MPC DESIGN

A. V-MPC Formulation in Space Domain

In this section, we propose V-MPC formulation to solve optimal problem. While pure RL could handle HEV energy management, MPC provides critical advantages: guaranteed constraint satisfaction and interpretable control behavior essential for automotive applications. However, MPC's key limitation is selecting appropriate terminal cost parameters, particularly λ . Our hybrid approach uses RL specifically to solve this parameter selection challenge while preserving MPC's constraint handling benefits. V-MPC formulation can be expressed as follows:

$$\min J = \sum_{k=0}^{N-1} \underbrace{\Delta m_{f,d}(x, u, k|d)}_{\text{Stage cost}} + \underbrace{\hat{V}(x(k+N)|d)}_{\text{Terminal cost}}, \quad (29a)$$

subject to

$$x(k+1|d) = x(k|d) + f(x(k|d), u(k|d))\Delta d, \quad (29b)$$

$$x_{\min} \leq x(k|d) \leq x_{\max}, \quad (29c)$$

$$u_{\min} \leq u(k|d) \leq u_{\max}, \quad (29d)$$

$$T_{e,\min} \leq T_e(k|d) \leq T_{e,\max}, \quad (29e)$$

$$T_{m,\min} \leq T_m(k|d) \leq T_{m,\max}, \quad (29f)$$

$$k = 0, 1, \dots, N-1, \quad (29g)$$

where $x = SOC$ is state, u is control input, $f(x, u)$ is the state function, N is prediction horizon and k is instant distance. We structure the optimal control problem by decomposing the objective function (29a) into stage and terminal cost components. MPC handles stage cost optimization with accessible short-term forecasting information. Terminal costs, calculated using VF in (28), are incorporated to assess the state value $x(k+N)$ for the remaining travel distance. VF allows MPC to balance fuel and battery consumption. The V-MPC controller utilized MATLAB's `fmincon` optimization solver.

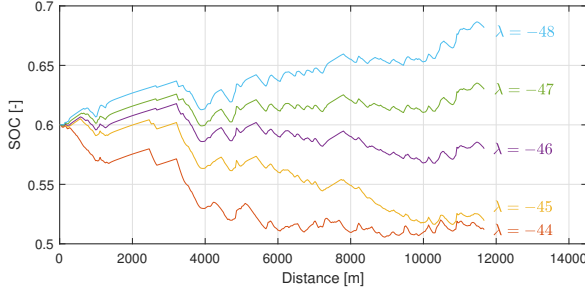


Fig. 5. SOC results according to VF value.

B. Value Function Approximation via Reinforcement Learning

In this work, we approximate the VF for a repeat route. According to $\hat{\lambda}$ in (28), VF manages energy consumption between fuel and battery in the V-MPC formulation in (29a). As demonstrated in Fig. 5, the weighting parameter λ critically affects SOC trajectory and energy management performance, making manual parameter selection extremely challenging for varying driving conditions. Following the theoretical framework [23], where optimal policies can be recovered through appropriate cost modification, we employ RL to learn the optimal $\hat{\lambda}$ that adapts to different scenarios, effectively solving this parameter tuning problem. The exact VF value can be obtained through the cost-to-go in equation (27), but this is only feasible when complete route information is available. Therefore, we derive the VF approximation value through RL utilizing similar speed patterns in repeated routes.

1) *MDP formulation*: We pose the VF learning on a finite-horizon markov decision process in the space domain. The state is $s := [SOC, d]^T \in \mathcal{S}$ and the action is $a := \hat{\lambda} \in \mathcal{A}$ obtained by discretizing a physically meaningful range $[\lambda_{\min}, \lambda_{\max}]$. Given (s, a) , the environment executes the V-MPC optimization in (29a), yields the control input sequence over the prediction horizon, advances the state to $s(d+1)$, and returns the scalar reward $r(d+1) := \mathcal{R}(s, a)$ as shown in Fig. 6. RL learns the optimal policy through interaction with the environment.

2) *State space*: Since RL learns optimal actions based on states, defining the state space is crucial and can be expressed as follows:

$$SOC \in \{SOC^1, SOC^2, \dots, SOC^m\} \in \mathbb{R}^m, \quad (30a)$$

$$d \in \{d^1, d^2, \dots, d^n\} \in \mathbb{R}^n, \quad (30b)$$

$$\mathcal{S} = \{(SOC^1, d^1), \dots, (SOC^m, d^n)\} \in \mathbb{R}^{m \times n}, \quad (30c)$$

where d denotes the travel distance and $\mathcal{S}$ represents the state space. Both the distance d and SOC are updated through V-MPC execution. The state $s \in \mathcal{S}$ consists of SOC and distance s , because $\hat{\lambda}$ depends on both variables in equation (27). This formulation captures the state VF that characterizes SOC's worth at each travel distance. Excessively large state spaces cause memory constraints, computational complexity, and exploration difficulties during learning. Therefore, function approximation with appropriate discretization is essential.

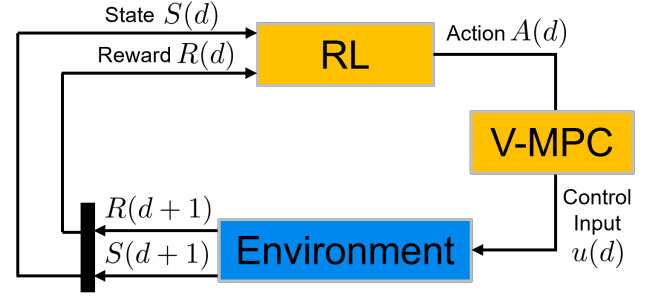


Fig. 6. VF approximation structure using V-MPC and RL.

SOC is discretized considering the state constraints of V-MPC, and d is discretized considering the terminal distance d_f .

3) *Action space*: The action space can be defined as follows:

$$\mathcal{A} = \{\hat{\lambda}^1, \hat{\lambda}^2, \dots, \hat{\lambda}^j\} \in \mathbb{R}^j, \quad (31)$$

where $\mathcal{A}$ is the action space with j intervals. The objective of RL is to find an action $\hat{\lambda}$, and it is crucial to learn the behavior that yields the highest reward when the action value is applied through V-MPC. VF evaluates the future state value using (28) after the state is computed by the stage cost. In this paper, the action range is constrained using VF to ensure the terminal SOC can be achieved within the feasible bounds. The action can be learned by maximizing the cumulative reward from the environment.

4) *Reward space*: Reward serves as a crucial signal that guides the learning direction in RL. When an agent takes an action, the environment provides numerical feedback indicating whether that action was good or bad. When the RL agent proposes a $\hat{\lambda}$ value, the MPC uses this value to distribute energy, and the environment calculates the reward based on the resulting performance. The reward function is defined as:

$$\mathcal{R}(s, a) = \dot{m}_{f,d} + \frac{C_b \cdot V_{oc}}{LHV_{fuel}} \Delta SOC, \quad (32)$$

where LHV_{fuel} is lower heating value. The reward function defined in (32) represents the total equivalent fuel consumption at each time step. The first term $\dot{m}_{f,d}$ denotes the actual fuel consumption rate, while the second term converts battery energy usage to fuel equivalent units. This approach allows the RL agent to minimize total energy use by considering both fuel and battery consumption.

5) *Bellman optimality and DDQN targets*: Various RL techniques exist and can be chosen based on the specific application field [18]. In this study, we utilize the double deep Q-learning (DDQL) algorithm to approximate the discretized VF, while comprehensive performance comparison among different RL algorithms is beyond the scope of this work. Let $Q^*(s, a)$ be the optimal action-value function with discount factor $\gamma \in (0, 1)$:

$$Q^*(s_d, a_d) = \mathbb{E} \left[r_d + \gamma \max_{a_{d+1} \in \mathcal{A}} Q^*(s_{d+1}, a_{d+1}) \mid s_d, a_d \right]. \quad (33)$$

Algorithm 1 DDQL Algorithm for Value Function Approximation

```

1: Initialize Q-network with weights  $\Theta$ , target network  $\Theta^- = \Theta$ 
2: Initialize replay memory  $\mathcal{D}$ , set hyperparameters
3: for  $i = 1$  to maximum episodes do
4:   Initialize state  $s_0^i$  and episode distance  $d = 0$ 
5:   while  $d < d_f$  do
6:     Choose action  $a_d^i = \hat{\lambda}$  using  $\varepsilon$ -greedy policy from
(37)
7:     Execute V-MPC with  $\hat{\lambda}$ , observe  $r_d^i$  and  $s_{d+1}^i$ 
8:     Store transition  $\{s_d^i, a_d^i, r_d^i, s_{d+1}^i\}$  in  $\mathcal{D}$ 
9:     Sample mini-batch  $\mathcal{M}$  from  $\mathcal{D}$ 
10:    Compute target:  $y = r + \gamma \max_{a'} Q(s_{d+1}; \Theta^-)$ ,
prediction:  $\hat{y} = Q(s_d, a_d; \Theta)$ 
11:    Update network:  $\Theta \leftarrow \Theta - \alpha \nabla_{\Theta}(y - \hat{y})^2$ 
12:     $d \leftarrow d + \Delta d$ 
13:  end while
14:  Update exploration:  $\varepsilon^{i+1}$ 
15:  Periodically update target network:  $\Theta^- \leftarrow \Theta$ 
16: end for

```

We approximate Q^* by a neural network $Q(s, a; \Theta)$ and employ Double DQN with a target network $Q(\cdot; \Theta^-)$ to mitigate overestimation. For a transition $\tau_i = (s_d^i, a_d^i, r_d^i, s_{d+1}^i)$ sampled from the replay buffer $\mathcal{D}$, the DDQN target is

$$y^i = r^i + \gamma Q\left(s_{d+1}^i, \arg \max_{a_{d+1} \in \mathcal{A}} Q(s_{d+1}^i, a_{d+1}^i; \Theta); \Theta^-\right), \quad (34)$$

where Θ is hyperparameter of neural network. The squared temporal-difference loss is

$$\mathcal{L}(\Theta) = \frac{1}{B} \sum_{i=1}^B \left(y^i - Q(s^i, a^i; \Theta)\right)^2, \quad (35)$$

where B is the minibatch size. We update Θ via gradient descent and periodically update the target network parameters by a soft copy

$$\Theta^- \leftarrow (1 - \rho) \Theta^- + \rho \Theta, \quad \rho \in (0, 1]. \quad (36)$$

When DDQL selects an action $\hat{\lambda} \in \mathcal{A}$, V-MPC solves equation (29a) with the given $\hat{\lambda}$. The system then updates to the next state $s + 1$ and computes the corresponding reward $r(s + 1)$, storing both in DDQL's memory buffer. DDQL determines the optimal action $\hat{\lambda}$ by maximizing cumulative reward through iterative interaction with environment.

6) *Policy and action selection*: The balance between exploration and exploitation is crucial for loss minimization and cumulative reward maximization. We implement the ε -greedy strategy in DDQL as follows:

$$\pi_{\varepsilon}(s) = \begin{cases} \text{random}(\mathcal{A}), & \text{with prob. } c < \varepsilon, \\ \arg \max_{a \in \mathcal{A}} Q(s, a; \Theta), & \text{otherwise,} \end{cases} \quad (37)$$

where π represents the policy, $c \in [0, 1]$ denotes a uniformly distributed random number, and $Q(\cdot)$ is the action-value function. The exploration parameter ε decays from 0.99 to 0.001

during training. When $c < \varepsilon$, an action $a \in \mathcal{A}$ is randomly selected from the action space for exploration. Otherwise, the action a that maximizes the Q-value $Q(\cdot)$ is chosen for exploitation.

The greedy policy used for deployment and for computing the DDQN target is

$$\hat{\lambda}(s) = \pi(s) := \arg \max_{a \in \mathcal{A}} Q(s, a; \Theta). \quad (38)$$

Algorithm 1 presents the DDQL framework for optimizing VF in the proposed V-MPC controller. The algorithm initializes a main Q-network with weights Θ and a target network Θ^- , along with an experience replay buffer $\mathcal{D}$. For each episode, the agent selects actions using an ε -greedy policy, executes the V-MPC controller, and stores the resulting transitions in the replay buffer. The Q-network is trained by sampling mini-batches from $\mathcal{D}$ and minimizing the temporal difference error between the target value in (34) and the predicted value $\hat{y} = Q(s_d, a_d; \Theta)$. The exploration rate ε is gradually decayed throughout training, and the target network is periodically updated to stabilize learning and mitigate overestimation bias. The DDQL algorithm was implemented in MATLAB using the Deep Learning Toolbox with `dlnetwork` for neural network construction and `dlfeval` for automatic differentiation, while neural network training employed the `adamupdate` function for parameter updates.

In this work, we consider the CS mode operation of PHEVs with limited battery capacity constraints. Assuming that the remaining travel distance is available from the onboard navigation system, VF can be formulated as follows:

$$\hat{V}(\cdot) = \begin{cases} \hat{\lambda}(s) \cdot (SOC(\cdot) - SOC_f), & \text{if } d \leq d_r \\ w \cdot (SOC(\cdot) - SOC_f)^2, & \text{otherwise} \end{cases} \quad (39)$$

where w is the weighting parameter and d_r is the threshold travel distance. Both d_r and w are tuning parameters. If the current travel distance is less than d_r , the approximated VF is utilized. The optimal action $\hat{\lambda}(s)$ is determined by mapping the current state according to the state space $\mathcal{S}$. This action $\hat{\lambda}(s)$ is then utilized during the V-MPC prediction horizon optimization process. Otherwise, the term $w \cdot (SOC(\cdot) - SOC_f)^2$ is incorporated into (29a) for CS mode operation.

TABLE II
DDQL TRAINING HYPERPARAMETERS

Category	Parameter	Value
Network Architecture	Hidden layer 1	40 neurons
	Hidden layer 2	30 neurons
	Hidden layer 3	20 neurons
	Activation function	Tanh
DDQL Training	Episodes	1000
	Discount factor	0.99
	Initial exploration	0.99
	Exploration decay	0.990
	Minimum exploration	0.001
	Initial learning rate	0.001
	Learning rate decay	0.999

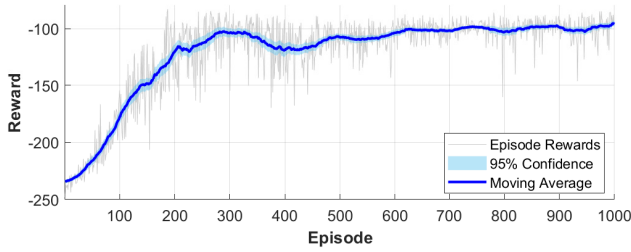


Fig. 7. Training rewards obtained using offline training data.

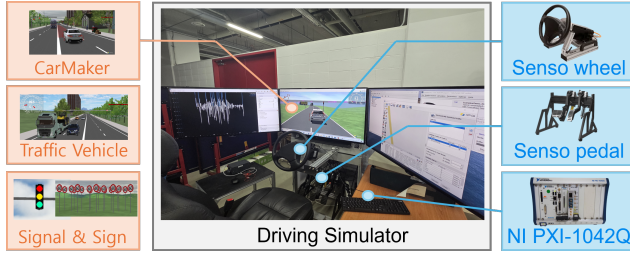


Fig. 8. Driving simulator experiment platform.

As shown in Table II, essential RL-related hyperparameters are listed. The hyperparameters listed in Table II were selected through empirical evaluation to achieve optimal reward convergence. The resulting DDQL training performance is illustrated in Fig. 7. The reward starts low during the initial exploration phase, then increases substantially as the agent shifts toward exploitation of learned policies.

V. SIMULATION RESULTS

A. Simulation Environment

In this section, we evaluate V-MPC on repeated routes using offline simulation. Offline simulation confirms that V-MPC solves short-term prediction problems and the VF reduces fuel consumption while maintaining CS mode. We first train the VF offline using a driving simulator as shown in Fig. 8. The speed data obtained from CarMaker software, which includes realistic traffic, signals, and road signs for repeated routes. We control the simulated vehicle using SensoWheel and SensoPedal devices, with steering and acceleration commands sent through NI PXI-1042Q hardware. To evaluate our proposed V-MPC, the following strategies are considered:

- DP: DP is considered as the benchmark for comparison.
- V-MPC: The approximated value function is obtained from the trained deep reinforcement learning network, providing terminal cost estimates for finite-horizon optimization.
- MPC: A fixed value function value is selected to prioritize CS mode operation without adaptive learning capability.
- RB: Rule-based control serves as the baseline strategy for online simulation to evaluate real-time application performance.
- ECMS*: The equivalence factor is optimized offline for each driving scenario using the shooting method, iterating

TABLE III
V-MPC PERFORMANCE ANALYSIS ACROSS DIFFERENT PREDICTION HORIZONS

Horizon (N)	8	10	12	14
Final SOC	0.6019	0.6022	0.6011	0.6013
Fuel Consumption [g]	601.18	595.58	592.34	588.76
Execution Time [ms]	1.67	2.82	3.89	5.13

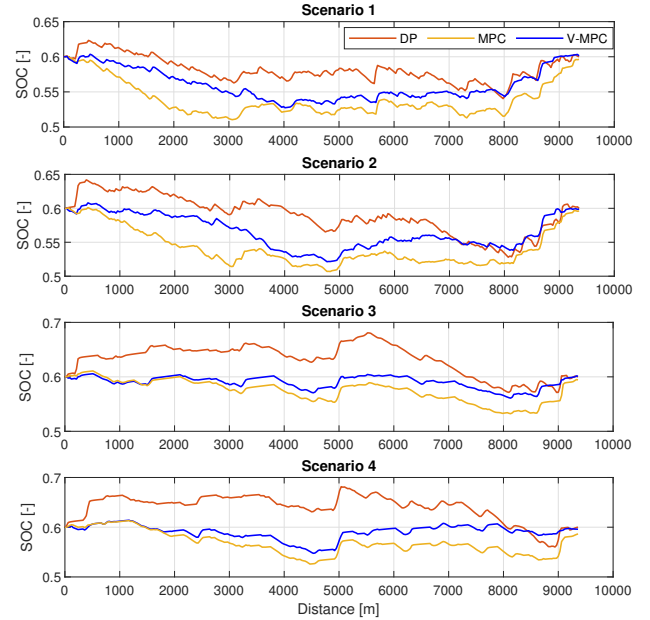


Fig. 9. SOC trajectories for different control methods in a randomly selected scenario from the test data.

until both fuel consumption minimization and charge-sustaining constraints are satisfied [29], [30].

- A-ECMS: Starting with an initial equivalence factor, the PI controller gains are systematically tuned to enable real-time adaptive adjustment based on SOC feedback [29], [30].

B. Influence of V-MPC Prediction-Horizon Length

Table III shows performance evaluation across different prediction horizons. Longer horizons consistently improve fuel economy, reducing consumption from 601.18 g (N=8) to 588.76 g (N=14) while maintaining similar SOC values (0.601-0.602). This improvement occurs because extended horizons allow the MPC to optimize using more actual stage costs rather than terminal cost approximation. Despite superior fuel economy at longer horizons, $N = 10$ represents an optimal balance between performance and practical robustness. While this study assumes accurate speed prediction, longer horizons increase dependency on forecasting accuracy in real-world applications, where prediction errors accumulate over extended time windows. The selected horizon $N=10$ achieves good fuel economy (595.58 g) while maintaining reasonable prediction reliability for practical implementation.

TABLE IV
COMPARATIVE PERFORMANCE ACROSS DRIVING SCENARIOS IN REPEATED ROUTE

Scenario	Control Strategy	Final SOC [-]	Fuel Consumption [g]	Fuel Consumption Difference [%]
1	DP	0.6000	502.4151	—
	MPC	0.5962	546.9342	+8.14
	V-MPC	0.6014	522.5649	+3.86
2	DP	0.6000	488.9855	—
	MPC	0.5989	544.7399	+10.24
	V-MPC	0.5989	518.5369	+5.70
3	DP	0.6000	467.2459	—
	MPC	0.5945	504.5050	+7.39
	V-MPC	0.6012	483.7733	+3.42
4	DP	0.6000	545.9856	—
	MPC	0.5865	588.4931	+7.22
	V-MPC	0.5960	563.7169	+3.15

C. Influence of Value Function Approximation

In this section, we analyze influence of VF approximation under repeated route. The approximated VF is derived through RL trained by collected driving data from Fig. 8. As shown in Fig. 9, SOC trajectories are depicted for various scenarios. Since DP utilizes complete driving information, it can calculate the optimal global solution compared to MPC and V-MPC approaches. DP maintains successfully $SOC_f = 0.6$ through the cost-to-go function. In contrast, MPC and V-MPC approximately achieve $SOC_f = 0.6$ using the proposed VF in (39). Assuming that the residual travel distance $d_r = 1000$ m is provided by the navigation system, both the MPC and V-MPC initiate battery charging for the operation of the CS mode.

Table IV presents a quantitative comparison of fuel consumption and SOC performance across four different driving scenarios. The results demonstrate that V-MPC achieves superior performance compared to conventional MPC when evaluated against the optimal DP benchmark. Specifically, V-MPC exhibits fuel consumption deviations ranging from 3.15% to 5.70% relative to the DP solution, while conventional MPC shows significantly larger deviations of 7.22% to 10.24%. This substantial improvement is attributed to the VF approximation capability of V-MPC, which enables more informed long-term state evaluation within the limited prediction horizon.

D. Influence of Unexpected Scenarios

This section evaluates the performance of V-MPC when applied to unexpected driving scenarios that are not included in the training dataset. V-MPC is trained on US06 and LA92 repeated route scenarios to learn VF approximation for route-specific energy management. Table V presents a performance comparison across five different unseen driving scenarios to evaluate the algorithm's generalization capability.

V-MPC generally outperforms conventional MPC in most scenarios, showing lower fuel consumption deviations from

TABLE V
COMPARATIVE PERFORMANCE OF UNEXPECTED DRIVING SCENARIOS

Scenario	Control Strategy	Final SOC [-]	Fuel Consumption [g]	Fuel Consumption Difference [%]
UDDS	DP	0.6009	472.0763	—
	MPC	0.6003	490.3815	+3.88
	V-MPC	0.6014	481.2681	+1.95
HWFET	DP	0.6070	727.9564	—
	MPC	0.5992	730.2579	+0.32
	V-MPC	0.5994	733.7090	+0.79
LA92	DP	0.6000	766.9729	—
	MPC	0.5963	776.8781	+1.29
	V-MPC	0.5963	771.6833	+0.61
LA92+HWFET	DP	0.6066	1502.4305	—
	MPC	0.5989	1526.1136	+1.58
	V-MPC	0.5979	1516.4247	+0.93
US06+US06H+LA92	DP	0.6034	2068.7544	—
	MPC	0.5967	2150.2444	+3.94
	V-MPC	0.5962	2084.3121	+0.75

optimal DP (averaging 1.01% vs MPC's 2.20%). This improvement stems from the learned VF through RL training, which provides more appropriate penalties for SOC deviation compared to MPC's constant terminal cost approximation.

However, the results reveal limitations in cross-scenario robustness. Notably, in the HWFET scenario, V-MPC consumes more fuel than conventional MPC, indicating that the VF learned from urban driving patterns not generalize effectively to steady highway conditions. While V-MPC excels in scenarios similar to its training data, developing truly robust energy distribution strategies for multi-scenario applications requires future research into enhanced generalization techniques and robustness considerations.

E. Real-time Simulation Results

In this section, we conduct real-time simulation using Autonomie software to evaluate V-MPC performance [10]. The simulation structure is shown in Fig. 10. The driver follows reference velocity from the driving simulator on repeated routes. The demanded torque is calculated according to current vehicle state, and V-MPC computes the optimal split ratio to minimize fuel consumption. During optimization, the value

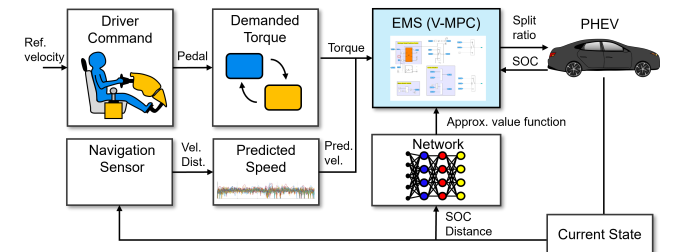


Fig. 10. Real-time simulation structure using Autonomie.

TABLE VI
COMPARATIVE PERFORMANCE OF REAL-TIME SIMULATION ACROSS
DRIVING SCENARIOS

No.	Strategy	Init SOC [-]	Final SOC [-]	Fuel Consump. [g]	Fuel Consump. Improvement [%]
1	RB	0.6000	0.5463	478.7	—
	ECMS*	0.6000	0.6000	498.2	-4.07
	A-ECMS	0.6000	0.6002	504.5	-5.38
	MPC	0.6000	0.5956	511.6	-6.87
	V-MPC	0.6000	0.5953	500.3	-4.52
2	RB	0.5463	0.5557	408.6	—
	ECMS*	0.6000	0.6001	398.5	+2.47
	A-ECMS	0.6002	0.5850	401.2	+1.81
	MPC	0.5956	0.5975	408.8	-0.05
	V-MPC	0.5953	0.6000	401.9	+1.64
3	RB	0.5557	0.5605	403.9	—
	ECMS*	0.6001	0.5980	355.9	+11.88
	A-ECMS	0.5850	0.6006	387.7	+4.01
	MPC	0.5975	0.5984	394.6	+2.30
	V-MPC	0.6000	0.5995	354.6	+12.20
4	RB	0.5605	0.5630	410.3	—
	ECMS*	0.5980	0.5996	392.1	+4.43
	A-ECMS	0.6006	0.5967	395.6	+3.58
	MPC	0.5984	0.5981	397.5	+3.11
	V-MPC	0.5995	0.6003	393.3	+4.14
5	RB	0.5630	0.5888	450.6	—
	ECMS*	0.5996	0.5992	421.1	+6.54
	A-ECMS	0.5967	0.6135	448.6	-0.66
	MPC	0.5981	0.5566	407.7	+9.52
	V-MPC	0.6003	0.6004	425.1	+5.66
6	RB	0.5888	0.5720	281.7	—
	ECMS*	0.5992	0.6014	278.9	+0.99
	A-ECMS	0.6135	0.5901	268.3	+4.75
	MPC	0.5566	0.6026	323.8	-14.94
	V-MPC	0.6004	0.6020	281.7	0.00
7	RB	0.5720	0.6209	312.7	—
	ECMS*	0.6014	0.6033	280.6	+10.26
	A-ECMS	0.5901	0.6058	290.4	+7.13
	MPC	0.6026	0.6041	292.6	+6.42
	V-MPC	0.6020	0.6050	280.6	+10.04
Total	RB	—	—	2746.5	—
	ECMS*	—	—	2625.3	+4.41
	A-ECMS	—	—	2696.3	+1.86
	MPC	—	—	2836.6	-3.28
	V-MPC	—	—	2637.5	+3.96

function is obtained from the neural network trained offline using RL. Testing covers seven days of repeated driving with daily learning updates. Each day's final SOC becomes the starting SOC for the next day. We compare RB, ECMS, A-ECMS, MPC, and V-MPC for real-time validation. DP is excluded because it cannot operate in real-time and requires complete route information.

Table VI shows the real-time simulation results across seven days of repeated driving. ECMS* achieved the best fuel economy with 4.41% improvement over the baseline RB strategy, representing the theoretical upper bound with perfect parameter tuning. Among practically implementable methods, V-MPC achieved 3.96% fuel reduction, approaching

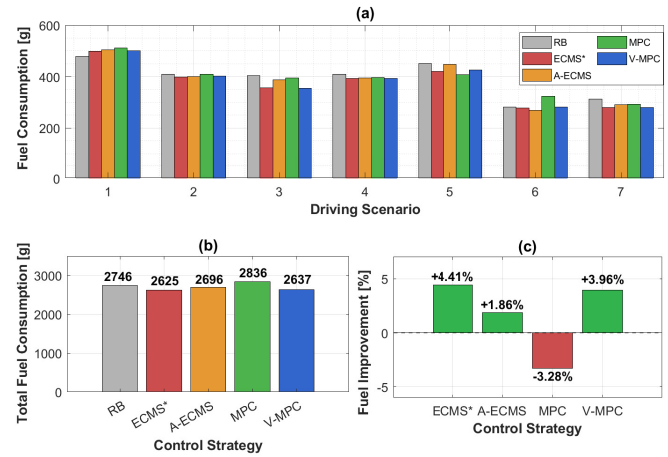


Fig. 11. Fuel consumption and improvement with real-time simulation. (a) fuel consumption, (b) total fuel consumption, (c) fuel improvement compared with RB.

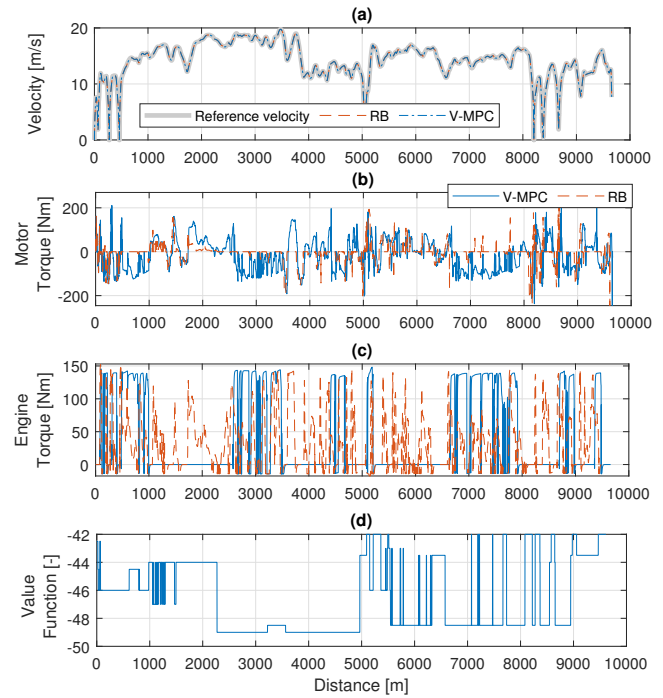


Fig. 12. Simulation results in Scenario 7. (a) velocity, (b) motor torque, (c) engine torque, (d) VF of V-MPC.

the optimal ECMS* performance of 4.41%. A-ECMS showed moderate improvement of 1.86%, while conventional MPC performed worse than baseline by 3.28% as shown in Fig. 11. The results confirm that V-MPC provides the best fuel economy among real-world applicable strategies while maintaining for real-time implementation.

As shown in Fig. 12, simulation results using Autonomie are depicted for Day 7. Both V-MPC and RB distribute the demanded torque obtained from driver commands such as deceleration and acceleration pedal inputs. As a result, both RB and V-MPC successfully follow the reference velocity as

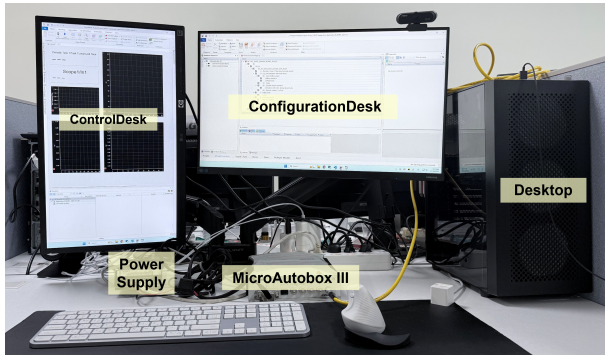


Fig. 13. Simulation environment and real-time implementation results using MicroAutoboxIII to check memory footprint and actual real-time implementability.

TABLE VII
REAL-TIME EXECUTION PERFORMANCE [ms]

	RB	A-ECMS	MPC	V-MPC
Min	0.023	0.024	0.613	0.577
Avg	0.025	0.117	2.825	2.349
Max	0.027	0.195	6.425	6.600

shown in Fig. 12(a). Additionally, V-MPC performs prediction horizon optimization incorporating the VF value as shown in Fig. 12(d). The VF is obtained from the trained network according to the current SOC and travel distance. The VF serves as a critical parameter for energy management, significantly impacting the motor and engine torque distribution as shown in Fig. 12(b) and (c).

F. Embedded Platforms Test

To address computational feasibility and memory footprint concerns, the proposed algorithm was implemented and tested on a MicroAutoboxIII (DS1403) real-time prototyping system as shown in Fig. 13. To assess real-time implementation feasibility, comprehensive testing was performed using embedded hardware deployment. The integrated system combining the vehicle model and control algorithm was transferred to ECU hardware for real-time signal validation [31], [32]. Communication between the development PC and embedded hardware was established using dSPACE's ConfigurationDesk and ControlDesk software tools. ConfigurationDesk enables the deployment of simulation models to real-time hardware platforms and facilitates their execution on dSPACE systems. The software provides graphical interfacing capabilities to connect Simulink-developed models directly to dSPACE hardware platforms.

Table VII presents the real-time execution performance analysis on MicroAutoboxIII. RB control demonstrates the fastest performance with an average execution time of 0.025 ms, while A-ECMS exhibits 0.117 ms average execution time. MPC-based approaches require higher computational resources: conventional MPC averages 2.825 ms and V-MPC averages 2.349 ms. Although these optimization-based approaches require longer execution times, all evaluated methods

TABLE VIII
MEMORY FOOTPRINT ANALYSIS [KB]

Section	RB	A-ECMS	MPC	V-MPC
.text	56.06	69.66	129.72	141.32
.data	461.71	461.94	511.59	511.61
.rodata	1.16	1.11	1.23	4.34
.bss	10.28	173.50	178.76	282.25
Total	529.21	706.20	821.30	939.53

remain computationally feasible for real-time energy management applications, where optimization horizons of several seconds are commonly employed in production vehicles.

Table VIII presents memory footprint analysis for the evaluated control strategies. RB control shows the lowest memory usage at 529.21 KB, while A-ECMS requires 33.4% additional memory. MPC-based approaches exhibit higher memory demands: conventional MPC consumes 55.2% more memory and V-MPC requires 77.5% more memory than the baseline. Despite V-MPC having the highest memory consumption at 939.53 KB, this represents less than 0.05% of the MicroAutoboxIII's 2 GB RAM capacity, demonstrating that all methods are feasible for embedded implementation.

G. Limitation

The proposed V-MPC framework assumes a repeated route, which is common in real-world applications such as commuting, delivery, and fixed transit routes. However, its applicability to arbitrary or unexpected driving scenarios remains limited, as the learned value function may not generalize beyond the training distribution. Specifically, the scalar value function $\hat{V}(\text{SOC}, d)$, parameterized solely by SOC and travel distance, neglects other relevant system states such as engine thermal dynamics and auxiliary power demands, potentially leading to suboptimal energy allocation when these factors significantly affect powertrain efficiency. Furthermore, as demonstrated in Section V-D, the approximation error increases under unexpected driving scenarios such as HWFET, where speed patterns deviate substantially from the training distribution. To address these limitations, future work should explore higher-dimensional value function approximations incorporating additional state variables, as well as adaptive learning strategies to improve robustness across diverse driving conditions.

VI. CONCLUSION

This paper presented V-MPC framework that addresses the prediction horizon limitations of conventional MPC in HEV energy management. The proposed approach incorporates RL to approximate complex VF using repeated driving route characteristics, eliminating the need for complete route information while maintaining near-optimal performance. Offline experiments show that V-MPC uses only 3.15-5.70% more fuel than the optimal DP solution, which is much better than conventional MPC that uses 7.22-10.24% more fuel. real-time experiments shows V-MPC works in real-time, reducing fuel consumption by 3.96% compared to RB control while

maintaining CS operation without speed tracking issues. Also, V-MPC shows real-time ability, with calculations completing in 0.66 seconds per second of simulation time. However, the RL approach faces robustness issues when driver behavior or traffic conditions cause unexpected speed pattern changes. Future work should collect more diverse speed data from repeated routes and develop probabilistic speed predictions. Additionally, incorporating stochastic elements into MPC optimization would improve the system's ability to handle uncertain driving conditions.

REFERENCES

- [1] Giovanni De Filippis, Basilio Lenzo, Aldo Sorniotti, Patrick Gruber, and Wouter De Nijs. Energy-efficient torque-vectoring control of electric vehicles with multiple drivetrains. *IEEE Transactions on Vehicular Technology*, 67(6):4702–4715, 2018.
- [2] Kang Shen, Xinyou Ke, Fan Yang, Weibo Wang, Cheng Zhang, and Chris Yuan. Numerical energy analysis of in-wheel motor driven autonomous electric vehicles. *IEEE Transactions on Transportation Electrification*, 9(3):3662–3676, 2023.
- [3] Sanghyeon Nam, Chulwoo Moon, Suyong Park, Byeongtae Lee, and Kyoungseok Han. Design and implementation of comprehensive thermal management verification model for electric vehicles operating in cold climates. *International Journal of Automotive Technology*, 25(1):47–59, 2024.
- [4] Xiangfei Wang, Suyong Park, and Kyoungseok Han. Energy-efficient speed planner for connected and automated electric vehicles on sloped roads. *IEEE Access*, 10:34654–34664, 2022.
- [5] Weida Wang, Xinghua Guo, Chao Yang, Yuanbo Zhang, Yulong Zhao, Denggao Huang, and Changle Xiang. A multi-objective optimization energy management strategy for power split hev based on velocity prediction. *Energy*, 238:121714, 2022.
- [6] Yigeng Huangfu, Peng Li, Shengzhao Pang, Chongyang Tian, Sheng Quan, Yonghui Zhang, and Jiang Wei. An improved energy management strategy for fuel cell hybrid vehicles based on pontryagin's minimum principle. *IEEE Transactions on Industry Applications*, 58(3):4086–4097, 2022.
- [7] Di Chen, Mike Huang, and Anna G. Stefanopoulou. Discrete mixed-integer shooting (dmis): Algorithm and application to plug-in hybrid electric vehicle energy management accounting for fuel cranking and actual powertrain efficiency maps. *IEEE Transactions on Control Systems Technology*, 31(1):221–234, 2023.
- [8] Fengqi Zhang, Xiaosong Hu, Reza Langari, and Dongpu Cao. Energy management strategies of connected hevs and phev: Recent progress and outlook. *Progress in Energy and Combustion Science*, 73:235–256, 2019.
- [9] Md Shahin Munsif and Hicham Chaoui. Energy management systems for electric vehicles: A comprehensive review of technologies and trends. *IEEE access*, 12:60385–60403, 2024.
- [10] Namdoo Kim, Michael Duoba, Namwook Kim, and Aymeric Rousseau. Validating volt phev model with dynamometer test data using autonomie. *SAE International Journal of Passenger Cars-Mechanical Systems*, 6(2013-01-1458):985–992, 2013.
- [11] Suyong Park, Duc Giap Nguyen, Jinrak Park, Dohee Kim, Jeong Soo Eo, and Kyoungseok Han. Deep neural network nmpc for computationally tractable optimal power management of hybrid electric vehicles. In *2024 American Control Conference (ACC)*, pages 1640–1645. IEEE, 2024.
- [12] Olle Sundstrom and Lino Guzzella. A generic dynamic programming matlab function. In *2009 IEEE control applications (CCA) & intelligent control (ISIC)*, pages 1625–1630. IEEE, 2009.
- [13] Kunhee Ryu, Bumsu Kim, Jehwi Yoo, Jinsung Kim, Jae Sung Bang, and Juhoon Back. Robust torque vectoring with desired cornering stiffness for in-wheel motor vehicles. *IEEE Access*, 11:133021–133033, 2023.
- [14] Suyong Park, Junghyo Kim, Duc Giap Nguyen, Minsoo Woo, Daekwang Kim, and Kyoungseok Han. Hierarchical nmpc for energy-efficient torque vectoring in four in-wheel motor electric vehicles. *Control Engineering Practice*, 164:106477, 2025.
- [15] Xiaodong Sun, Mingzhou Xue, Yingfeng Cai, Xiang Tian, Zhijia Jin, and Long Chen. Adaptive ecms based on ef optimization by model predictive control for plug-in hybrid electric buses. *IEEE Transactions on Transportation Electrification*, 9(2):2153–2163, 2022.
- [16] Duc Giap Nguyen, Suyong Park, Jinrak Park, Dohee Kim, Jeong Soo Eo, and Kyoungseok Han. An mpc approximation approach for adaptive cruise control with reduced computational complexity and low memory footprint. *IEEE Transactions on Intelligent Vehicles*, 9(2):3154–3167, 2023.
- [17] Julakha Jahan Jui, Mohd Ashraf Ahmad, MM Imran Molla, and Muhammad Ikram Mohd Rashid. Optimal energy management strategies for hybrid electric vehicles: A recent survey of machine learning approaches. *Journal of Engineering Research*, 12(3):454–467, 2024.
- [18] Chunyang Qi, Yiwen Zhu, Chuanxue Song, Guangfu Yan, Feng Xiao, Xu Zhang, Jingwei Cao, Shixin Song, et al. Hierarchical reinforcement learning based energy management strategy for hybrid electric vehicle. *Energy*, 238:121703, 2022.
- [19] Zheng Chen, Hongji Gu, Shiquan Shen, and Jiangwei Shen. Energy management strategy for power-split plug-in hybrid electric vehicle based on mpc and double q-learning. *Energy*, 245:123182, 2022.
- [20] Dennis Kibalama, Yuxing Liu, Stephanie Stockar, and Marcello Canova. Model predictive control for automotive climate control systems via value function approximation. *IEEE Control Systems Letters*, 6:1820–1825, 2021.
- [21] Youyi Chen, Kyoung Hyun Kwak, Jaewoong Kim, Dewey Dohoy Jung, and Youngki Kim. Integrated thermal management of electric vehicles based on model predictive control with approximated value function. *IEEE Access*, 2024.
- [22] Jure Soldo, Branimir Škugor, and Joško Deur. Model predictive control of a parallel plug-in hybrid electric vehicle relying on dynamic programming and extended backward-looking model. *IEEE transactions on control systems technology*, 32(2):581–594, 2023.
- [23] Sébastien Gros and Mario Zanon. Data-driven economic nmpc using reinforcement learning. *IEEE Transactions on Automatic Control*, 65(2):636–648, 2019.
- [24] Suyong Park, Solyeon Kwon, and Kyoungseok Han. RL-mpc framework for data-driven energy management in mild hybrid electric vehicles. *IFAC-PapersOnLine*, 59(3):85–90, 2025.
- [25] Yanzhao Jia, Rabee Jibrin, Yutaro Itoh, and Daniel Görges. Energy-optimal adaptive cruise control for electric vehicles in both time and space domain based on model predictive control. *IFAC-PapersOnLine*, 52(5):13–20, 2019.
- [26] Fengqi Zhang, Xiaosong Hu, Teng Liu, Kanghui Xu, Ziwen Duan, and Hui Pang. Computationally efficient energy management for hybrid electric vehicles using model predictive control and vehicle-to-vehicle communication. *IEEE Transactions on Vehicular Technology*, 70(1):237–250, 2020.
- [27] Namwook Kim, Sukwon Cha, and Hui Peng. Optimal control of hybrid electric vehicles based on pontryagin's minimum principle. *IEEE Transactions on control systems technology*, 19(5):1279–1287, 2010.
- [28] Hoseinali Borhan, Ardalan Vahidi, Anthony M Phillips, Ming L Kuang, Ilya V Kolmanovsky, and Stefano Di Cairano. Mpc-based energy management of a power-split hybrid electric vehicle. *IEEE Transactions on Control Systems Technology*, 20(3):593–603, 2011.
- [29] Simona Onori, Lorenzo Serrao, and Giorgio Rizzoni. Adaptive equivalent consumption minimization strategy for hybrid electric vehicles. In *Dynamic systems and control conference*, volume 44175, pages 499–505, 2010.
- [30] Simona Onori, Lorenzo Serrao, and Giorgio Rizzoni. *Hybrid electric vehicles: Energy management strategies*, volume 13. Springer, 2016.
- [31] Vigneshwaran Palanirasan and P Sathishkumar. Enhancing vehicle roll and yaw stability through a novel semi-active interconnected structure mechanism using 4/3 hydraulic control valves. *Smart Materials and Structures*, 34(9):095009, 2025.
- [32] Marco Adami, Matteo Massaro, Umberto Montanaro, Pietro Stano, Carmine Caponio, and Davide Tavernini. A reinforcement learning motor torque modulation approach for enhancing occupants comfort on electric vehicles. *IEEE Access*, 13:202893–202905, 2025.



Suyong Park received the B.S. degree from the School of Mechanical Engineering, Yeungnam University, Gyeongsan, South Korea in 2021, and M.S. degree from the Department of Mechanical Engineering, Kyungpook National University, Daegu, South Korea in 2024, respectively. He is currently working toward the Ph.D. degree in the Department of Automotive Engineering, Hanyang University, Seoul, South Korea. His research interests include energy-efficient control, optimization theory, and reinforcement learning.



Solyeon Kwon received her B.S. and M.S. degrees in mechanical engineering from Kyungpook National University, Daegu, Korea in 2020 and 2023, respectively, and is currently pursuing a Ph.D. degree in automotive engineering from Hanyang University, Seoul, Korea. Her current research interests include vehicle dynamics and control, optimization, control theory, neural networks, and data-driven control.



Youngki Kim received his B.S. and M.S. degrees from Seoul National University, South Korea, in 2001 and 2003, respectively, and his Ph.D. degree from the University of Michigan, Ann Arbor, MI, USA, in 2014, all in mechanical engineering. He is currently an Associate Professor with the Department of Mechanical Engineering, University of Michigan-Dearborn (UM-Dearborn), Dearborn, MI, USA. Prior to joining UM-Dearborn, he held the position of a Research Engineer with Ann Arbor Technical Center, Southwest Research Institute, from 2015 to 2017. From 2003 to 2008, he was a Research Engineer with Hyundai-Kia Motor Company. His research interests include modeling and estimation/control of electrified powertrain systems, connected and automated vehicles, and energy storage systems. He was a recipient of the SAE Russell S. Springer award (2019) from SAE International. He is an Associate Editor of IEEE Transactions on Transportation Electrification.



Kyoungseok Han received his B.S. degree in Civil Engineering with a minor in Mechanical Engineering from Hanyang University, Seoul, South Korea, in 2013, followed by M.S. and Ph.D. degrees in Mechanical Engineering from KAIST, Daejeon, South Korea, in 2015 and 2018, respectively. He is currently an Associate Professor in the Department of Automotive Engineering at Hanyang University, Seoul, South Korea. Prior to this, he worked as a Postdoctoral Research Fellow at the University of Michigan from 2018 to 2020 and as an Associate Professor at Kyungpook National University, Daegu, South Korea from March 2020 to August 2024. His research interests include autonomous vehicle modeling and control, energy-efficient control of electric vehicles, reinforcement learning, and optimal control theory and its applications.